

Eight Verification Conditions for MRDTs: a mechanized, corrected soundness metatheory for RA-linearizability under three-way merge, with nine production data types discharged

Sal metatheory note
FP Launchpad, IIT Madras

July 3, 2026

Abstract

Mergeable replicated data types (MRDTs) resolve concurrent divergence by a *three-way* merge `mergeL(l, a, b)`: the two divergent states a, b are reconciled relative to their lowest common ancestor l in a git-like version DAG. The Neem methodology reduces RA-linearizability of an MRDT to a fixed catalogue of per-data-type verification conditions (VCs), tied together by a once-and-for-all soundness meta-theorem. A companion note¹ shows that the meta-theorem’s proof is unsound as written and that its VC catalogue is insufficient; this note lifts the corrected architecture to the *ternary* setting the paper actually claims. The lift is not routine: the LCA argument is load-bearing (the counter MRDT is provably outside the binary theorem), the paper’s own LCA lemma has a gap in its proof, and the lattice-flavoured repair that works for CRDTs provably does not survive (no idempotence, no usable order). What survives is a *delta* discipline: three unconditional relational VCs on `rc`, one merge symmetry, three *feasibility-conditioned* redistribution laws, and a single contextual *causal-delta* equation — eight VCs in all — from which every reachable configuration of the replicated store is RA-linearizable, *per version*. All of it is mechanized in Lean 4 with zero **sorry**s, and the VC set is validated in the strongest currency available: nine of the twelve production MRDTs of the Sal repository — including OR-sets, an enable-wins flag, counters, a tombstone RGA and Peritext — carry kernel-checked end-to-end RA-linearizability theorems through it.

Contents

1	Introduction	2
2	The ternary setting	3
3	Three facts the lift must respect	4
3.1	The LCA argument is load-bearing	4
3.2	The LCA lemma is a reachability invariant — with a gap in the published proof	5
3.3	The binary defeater lifts to a fully LCA-legal execution	5
4	Canonical states and the ternary Join Lemma	5

¹*Set-relative linearization and the Join Lemma* (`joinpeel-note.pdf`, same directory), which treats the binary (CRDT) specialization and develops the corrected proof architecture reused here.

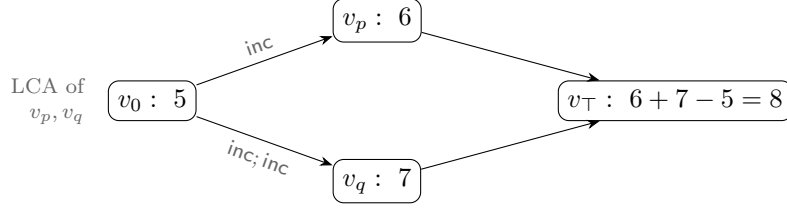


Figure 1: The counter MRDT. Both branches inherit the 5 from the fork point; the LCA argument lets the merge count each branch’s *delta* exactly once. No binary merge of 6 and 7 alone can recover 8: the information “how much is shared” lives in the DAG, and the LCA argument is how the data type reads it.

5	Why the contract is a delta contract	6
6	The eight verification conditions	7
7	Adequacy	9
8	The catalogue, discharged	10
9	Open questions	11
10	Mechanization	11

1 Introduction

An MRDT execution looks like a git history: replicas fork, apply operations locally, and merge. Unlike a state-based CRDT, whose binary merge must reconcile two states with no memory of their common past, an MRDT’s merge receives a third argument — the state at the *lowest common ancestor* (LCA) of the two versions being merged — and may use it to compute *deltas*: what each side did since the fork. The paradigmatic example is the counter,

$$\text{mergeL}(l, a, b) = a + b - l,$$

which adds the two sides’ increments-since- l instead of double-counting their common past (Figure 1).

The Neem methodology (OOPSLA 2025) promises a clean division of labour: the data-type designer discharges a fixed catalogue of equations over **do**, **mergeL** and the conflict-resolution relation **rc** — mostly by SMT — and a soundness meta-theorem, proved once, converts those VCs into *RA-linearizability* of every reachable configuration. The companion note documents what a Lean 4 mechanization found in the binary specialization: the meta-theorem’s central induction is unsound as written, and the VC catalogue is insufficient — there are reachable, VC-satisfying executions that are not RA-linearizable. It also develops the repair: a set-relative linearization order, canonical states $\sigma(E)$, and a *Join Lemma* whose induction consumes a small, contextual VC surface.

This note is about the setting the paper actually claims: **ternary merge over a version DAG**. Its contributions:

1. **The ternary lift is not an inert third argument (§3.1):** the counter satisfies the ternary shared-peel VC but *provably violates* its binary counterpart, so the class of MRDTs covered by the ternary theorem strictly exceeds anything the binary theorem can reach. “A CRDT is an MRDT that ignores its LCA” is a one-way instantiation, not an equivalence.
2. **The paper’s LCA lemma is true but its proof has a gap (§3.2):** $L(v_\top) = L(v_1) \cap L(v_2)$ holds, but only as a *reachability invariant* of the store, whose proof needs a generator-uniqueness induction the paper silently assumes. We mechanize the transition system and the invariant. The binary defeater configuration also lifts, verbatim, to a fully LCA-legal ternary execution (§3.3).
3. **The lattice repair provably does not survive the lift (§5):** MRDT merges are not idempotent ($\text{mergeL}(l, a, a) = 2a - l$ for the counter), the induced orders degenerate, and the honest associativity law has *four distinct LCAs* and holds only on feasible tuples. What replaces the lattice laws is a two-law *delta contract* — redistribution identities in which the LCA slot does, by arithmetic, the job idempotence did order-theoretically.
4. **Eight VCs suffice (§6–§7):** three relational VCs on rc (one of them carrying a different-replica guard that restores the paper’s own F^* artifact interface — the Lean transcription had dropped it, and a real data type detects the drop), merge symmetry, three feasibility-conditioned delta laws, and one contextual *causal-delta* equation. The adequacy theorem is per *version*: every version in the store, LCAs included, linearizes.
5. **The VC set is validated on the production catalogue (§8):** nine of the twelve MRDTs shipped in Sal — OR-set, an optimized OR-set, enable-wins flag, grow-only set and map, increment-only and PN counters, a tombstone RGA, and Peritext — are discharged end-to-end, kernel-checked. The sweep produces an empirical *class map* of which data types need which strength of contract, with two surprises: tombstones are a metatheoretic trivializer, and commutativity class and delta class are independent axes.

Everything stated as a theorem below is mechanized with zero **sorrys**; §10 maps statements to Lean names and files.

2 The ternary setting

Signature. An MRDT is $\langle \Sigma, \sigma_0, \text{do}, \text{mergeL}, \text{rc} \rangle$: a state space with initial state, an update function applied to *events* $e = (t, r, o)$ (a globally fresh timestamp, an origin replica, an operation), a ternary merge, and a conflict-resolution relation rc on operations used to order concurrent non-commuting pairs. We write $e(\sigma)$ for $\text{do}(\sigma, e)$, and $e_1 \rightleftharpoons e_2$ when $e_1(e_2(\sigma)) = e_2(e_1(\sigma))$ for every σ .

Executions. A configuration carries a *store*: a DAG of versions, each holding a state and the set of events that produced it, with per-replica heads. Transitions fork a replica, apply a fresh event at a head, query, or **merge**: pick heads v_1, v_2 , find their LCA $v_\top$ in the DAG, and install a new version with state $\text{mergeL}(\sigma_{v_\top}, \sigma_{v_1}, \sigma_{v_2})$ and event set $L(v_1) \cup L(v_2)$. Visibility $\xrightarrow{\text{vis}}$ records what each event’s issuing replica had seen.

RA-linearizability, per version. A configuration is RA-linearizable when *every version in the store* — not only the replica heads; LCAs are historical versions and the merge reads them — admits a sequencing π of exactly its event set such that π extends the linearization order lo and folds from σ_0 to the version’s state. The order lo is the paper’s: e_1 before e_2 if either $e_1 \xrightarrow{\text{vis}} e_2$ and they do not commute, or they are concurrent, do not commute, rc orders them, and e_2 is not already *absorbed* (overwritten) by a later non-commuting event. The companion note shows this order must be read *relative to the event set being linearized* (lo^E); we reuse that machinery wholesale.

Running examples. Three data types exercise every corner of the theory:

- the **counter**: $\Sigma = \mathbb{Z}$, $\text{inc}(\sigma) = \sigma + 1$, $\text{mergeL}(l, a, b) = a + b - l$; all operations commute, but the merge genuinely uses its LCA;
- the **OR-set** (tagged add-wins set): elements are tagged by the adding event’s timestamp; rem deletes the tags it has seen; concurrently, $\text{rem} \xrightarrow{\text{rc}} \text{add}$ resolves in favour of the add. The merge keeps an element iff *both* sides have it or *some* side added it since the fork:

$$\text{mergeL}(l, a, b) = (a \cap b) \cup (a \setminus l) \cup (b \setminus l)$$

— pointwise, “if the tag is in l , require it in both branches (nobody deleted it); otherwise one branch suffices (somebody added it).” Here the LCA acts as a *tombstone set*: deletion is represented by *absence relative to l* , so the state itself needs no graveyard;

- the **enable-wins flag**: per-replica pairs (counter, flag); enabling bumps your own counter and sets your flag; disabling clears all flags; the merge compares each replica’s counter against the LCA’s to decide whether an enable “happened since the fork” on that replica. Its merge *compares* counters rather than accumulating sets — and that difference will be visible in the metatheory (§8).

3 Three facts the lift must respect

3.1 The LCA argument is load-bearing

The paper notes that CRDTs are the special case “ignore the LCA argument.” The converse direction fails, mechanically:

Theorem 3.1 (the counter is outside the binary theorem). *The counter satisfies the ternary shared-peel VC*

$$\text{mergeL}(le, ae, be) = \text{mergeL}(l, a, b) \cdot e \quad (\text{for the counter: } (a+1)+(b+1)-(l+1) = (a+b-l)+1)$$

but provably violates the corresponding binary VC $\text{merge}(a \cdot e, b \cdot e) = \text{merge}(a, b) \cdot e$ for every LCA-forgetting binary collapse. *Counter_binary_lem_Oop_false*

So the ternary metatheory cannot be obtained by instantiating a binary one; it must be built natively, with the LCA state threaded through every merge-side statement. Concretely, in the corrected theory the shared peel becomes *three-sided*: a shared event is (by the LCA lemma below) exactly an LCA event, and peeling it removes it from all three components in lock-step.

3.2 The LCA lemma is a reachability invariant — with a gap in the published proof

Everything above silently used:

Lemma 3.2 (LCA lemma). *In every reachable configuration, if $v_\top$ is an LCA of versions v_1, v_2 in the store’s DAG, then $L(v_\top) = L(v_1) \cap L(v_2)$.*

This is what lets merge VCs be stated over *event sets*: the state the merge receives as l is the canonical state of exactly the intersection of the two sides’ histories. The lemma is true — but it is a *global induction over the reachable store*, not a local fact. Its mechanized proof threads a store invariant (all parents allocated; event sets monotone along edges; and an *origin* invariant — each event appears first at a unique generator version) through every transition; the published proof asserts the generator-uniqueness step without the induction that sustains it. An erratum-sized gap: the statement survives, the proof as written does not. (A related boundary fact, discovered while probing: *criss-cross* merges can defeat the *existence* of an LCA — they gate merge-enabledness, not the lemma.)

3.3 The binary defeater lifts to a fully LCA-legal execution

The companion note’s central counterexample — a reachable four-event add-wins configuration on which *every* bottom-up peel is stuck — might be hoped ternary-proof: perhaps the version DAG’s discipline rules it out. It does not. One staging replica realizes the intersection as an honest version, and the paper’s ternary merge induction breaks exactly as the binary one did.

Example 3.3 (the ternary defeater). *One key, add-wins semantics with tags, LCA-blind merge (a legal MRDT). Replica p adds (A_p) , replica q adds (A_q) . A staging replica s merges both one-add versions, creating v_s with $L(v_s) = \{A_p, A_q\}$. Now p removes (R_p) , seeing only A_p and then merges with v_s ; symmetrically q removes (R_q) and merges with v_s . Figure 2 shows the DAG. The final merge of the two heads finds $\text{LCA} = v_s$, and $L(v_s) = \{A_p, A_q\} = L(\text{head}_p) \cap L(\text{head}_q)$ — the configuration is fully legal. But exactly as in the binary analysis: every state-correct linearization of p ’s head ends in A_q (not R_p), every linearization of q ’s head ends in A_p (not R_q), while every linearization of the merged version must end in one of the removes. The events the induction must peel are buried in the very sides that own them; no ordering of the paper’s bottom-up rules applies.*

The consequences for the design are the same as binary, one level up: the metatheorem must be rebuilt on canonical states and a Join Lemma, and its merge-side VCs must be statements *about the join*, not about how either side’s state factors.

4 Canonical states and the ternary Join Lemma

The companion note’s repair carries over verbatim on the update side — mechanically so: the replica-facing core of a ternary configuration *is* a binary configuration by projection, so the set-relative order lo^E , its convergence theorem (any two lo^E -respecting enumerations of E fold to the same state), and hence **canonical states** $\sigma(E)$ are literally reused, not re-proved. The one genuinely ternary statement is what the merge must produce:

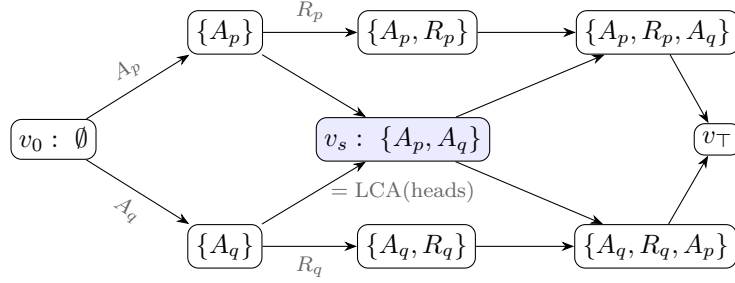


Figure 2: The ternary defeater (Example 3.3). Nodes are versions labelled by their event sets; the shaded version v_s , created by a staging replica, realizes $L(\text{head}_p) \cap L(\text{head}_q)$ as an honest store version, so the final merge is LCA-legal. The removes R_p, R_q — the only events a bottom-up derivation may peel last — are buried mid-history on their own sides.

Definition 4.1 (ternary Join Lemma). *For backward-closed event sets E_1, E_2 of a reachable configuration,*

$$\sigma(E_1 \cup E_2) = \text{mergeL}(\sigma(E_1 \cap E_2), \sigma(E_1), \sigma(E_2)).$$

By Lemma 3.2 the first argument is exactly the state the store hands the merge. The adequacy proof (§7) is then an induction over the transition system maintaining “every version holds the canonical state of its event set”; the merge case is the Join Lemma. What remains is the question this note exists to answer: which per-data-type VCs make the Join Lemma true?

5 Why the contract is a delta contract

In the binary world the answer was lattice-flavoured: associativity, idempotence and update-inflation, plus one contextual bound. Each of the three provably fails to survive the lift.

No idempotence. $\text{mergeL}(l, a, a) = 2a - l$ for the counter: only feasible *triples* (where l is the canonical state of the intersection — here, $l = a$) are idempotent. Unconditional idempotence would exclude the counter, i.e. exclude precisely the data types the ternary theorem exists for.

No usable order. The binary route’s workhorse order $x \sqsubseteq y := \text{merge}(x, y) = y$ has ternary candidates $x \sqsubseteq_l y := \text{mergeL}(l, x, y) = y$, but for the counter this relation degenerates to $x = l$; there is no antisymmetry left to split proof obligations into a “free half” and a bounded half. The single contextual VC below is accordingly an *equation*, not an inequality.

Associativity is the wrong target. The natural guess — $\text{mergeL}(l, \text{mergeL}(l, a, b), c) = \text{mergeL}(l, a, \text{mergeL}(l, b, c))$ — assumes one LCA for all three merges. In an execution, re-associating a three-way reconciliation involves *four* distinct LCAs:

$$\text{mergeL}(l_{(ab)c}, \text{mergeL}(l_{ab}, a, b), c) \stackrel{?}{=} \text{mergeL}(l_{a(bc)}, a, \text{mergeL}(l_{bc}, b, c)),$$

with $l_{ab} = \sigma(E_a \cap E_b)$, $l_{(ab)c} = \sigma((E_a \cup E_b) \cap E_c)$, and so on. For the counter — whose canonical states are additive measures over event sets — the two sides come to $a + b + c$ minus the two LCA measures, and they agree because

$$(E_a \cup E_b) \cap E_c = (E_a \cap E_c) \cup (E_b \cap E_c) \implies \text{both LCA-sums} = |E_{ab}| + |E_{ac}| + |E_{bc}| - |E_{abc}|$$

by inclusion–exclusion. The law holds, but only *through the set-algebra of intersections*: over four unconstrained states it is false. Honest associativity is inherently a feasible-tuple fact — and, the mechanization’s pleasant surprise, **the corrected induction never needs it**. Every merge the induction forms already sits at its honest LCA; the only laws consumed are two *redistribution* identities that equate two honestly-formed merges:

Definition 5.1 (the delta contract).

$$\begin{aligned} \text{LOCAL-REDISTRIBUTE: } \text{mergeL}(l, \text{mergeL}(m, x, c), y) &= \text{mergeL}(m, \text{mergeL}(l, x, y), c), \\ \text{REDISTRIBUTE: } \text{mergeL}(\text{mergeL}(m, x_0, c), \text{mergeL}(m, x_1, c), \text{mergeL}(m, x_2, c)) \\ &= \text{mergeL}(m, \text{mergeL}(x_0, x_1, x_2), c). \end{aligned}$$

Read c as a *delta relative to m* (in the induction, c will be “apply event e to its causal past”). LOCAL-REDISTRIBUTE says a delta application commutes past an enclosing merge through one branch slot. REDISTRIBUTE says a delta carried by *all three* components extracts *once* — the LCA slot cancels the duplicates by arithmetic. This is the delta contract’s quiet coup: cancelling the duplicated shared contribution is exactly the job idempotence performed in the binary theory, now done without any idempotence — which is why the counter sails through (REDISTRIBUTE for the counter is the identity $\sum x_i + 3c - 3m - (x_0 + c - m) - \dots$ collapsing on both sides).

Where the contract holds unconditionally — and where it cannot. The two laws hold *on all states* exactly for the group-like class (the counter) and the lattice-like, LCA-blind class (grow-only structures). They are *unconditionally false* for the OR-set: take an element tag present in c and l but in neither branch — the two sides of LOCAL-REDISTRIBUTE then disagree about whether the LCA’s copy survives. But such a tuple can never arise: a tag in a canonical LCA state is in *both* branches unless removed, and a removal is itself an event of the branch. The failing tuples are *infeasible*, and this is a theorem-grade phenomenon, not an inconvenience:

Finding. *For LCA-sensitive MRDTs beyond the group $\oplus$ lattice classes, the merge-coherence laws are irreducibly feasibility-conditioned: true on canonical tuples at honest LCAs, false in general. Quantifying merge VCs over all states — the published catalogue’s style — is not merely inconvenient for such data types; for several laws it is wrong in principle. (The enable-wins flag even falsifies the unit law $\text{mergeL}(\sigma_0, \sigma_0, s) = s$ on an infeasible state with a set flag and a zero counter.)*

Accordingly the delta laws enter the final contract in feasibility-conditioned form: quantified over canonical states of backward-closed event sets, with every **mergeL** node at its honest LCA. The unconditional form remains available as a one-line *on-ramp* (it trivially implies the conditioned form) for the group and lattice classes.

6 The eight verification conditions

For an MRDT $\langle \Sigma, \sigma_0, \text{do}, \text{mergeL}, \text{rc} \rangle$, the contract is:

Update layer (unconditional, SMT-shaped).

1. **rc-non-comm (guarded)** — for distinct events *from different replicas*: they fail to commute iff rc orders them in exactly one direction. *The conflict table is the commutativity*

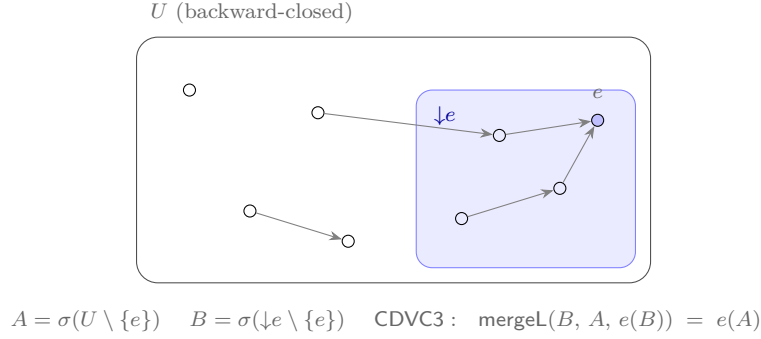


Figure 3: The causal-delta equation. The shaded region is e 's causal past $\downarrow e$ (what e saw); e is lo^U -maximal. e 's effect on the whole world A equals: compute e 's effect on its own causal past B , then merge that delta into A over LCA B . The merge sits at its honest LCA: $(U \setminus \{e\}) \cap \downarrow e = \downarrow e \setminus \{e\}$. Concurrent context that e never observed cannot amplify what e does.

table, with a direction. The different-replica guard matters: it is the F^* artifact's own interface form (`App_mrdt.fsti: requires distinct_ops o1 o2 /\ get rid o1 <> get rid o2`), dropped in an earlier transcription — and the *optimized OR-set* detects the drop: its same-replica same-element adds do not commute (the newer add evicts the older tag) yet are never concurrent, so `rc` rightly ignores them. Same-replica pairs are ordered by program order, not by conflict resolution.

2. **no-rc-chain** — conflict priorities do not chain: never $o_1 \xrightarrow{\text{rc}} o_2$ and $o_2 \xrightarrow{\text{rc}} o_3$. This is what makes the linearization order acyclic, so maximal events exist — the entire peel-and-recurse architecture stands on it.
3. **cond-comm** — a resolved conflict is invisible once an absorber runs: swapping an `rc`-ordered concurrent pair deep in a fold changes nothing for any later non-commuting event, across arbitrary intervening events. This is the engine of convergence.

Merge layer.

4. **merge symmetry** — $\text{mergeL}(l, a, b) = \text{mergeL}(l, b, a)$.
5. **feasible unit** — $\text{mergeL}(\sigma_0, \sigma_0, \sigma(E)) = \sigma(E)$: merging over nothing is a no-op, on states that can arise.
6. **feasible local-redistribute** and
7. **feasible redistribute** — Definition 5.1, quantified over canonical tuples at honest LCAs.
8. **the causal-delta equation (CDVC3)** — for a lo^U -maximal event e of a backward-closed U , with $A = \sigma(U \setminus \{e\})$ and $B = \sigma(\downarrow e \setminus \{e\})$ the canonical state of e 's own causal past:

$$\text{mergeL}(B, A, e(B)) = e(A).$$

What an event does is determined by what it saw (Figure 3): applying e to the whole world equals computing e 's delta on its causal past and merging it in over that past.

Items 1–4 are one-liners for every data type we have discharged; items 6–7 frequently hold unconditionally anyway (for the OR-set, REDISTRIBUTE is a pointwise Boolean tautology); essentially the entire per-data-type proof effort concentrates in item 8, whose discharge follows a uniform recipe: characterize $\sigma(E)$ (a “sandwich” invariant along canonical enumerations), then a *trichotomy* placing every event the maximal e interacts with either inside $\downarrow e$ or absorbed on a side that owns it.

Compare the published catalogue: 24 VCs per data type, plus five more the soundness proof uses silently. Seventeen of the 24 — the BOTTOMUP induction-scheme families — exist to drive the broken induction and are consumed nowhere in the corrected one; the paper’s MERGEIDEMPOTENCE is likewise never used. The seventeen were, however, doing honest work of a different kind: they are an *automation layer*, Skolemizing “holds on feasible states” into base-and-step equations an SMT solver can discharge. Rebuilding that layer, aimed at the corrected target (CDVC3 rather than BOTTOMUP), is in our view the most valuable open engineering problem this work leaves (§9).

7 Adequacy

Theorem 7.1 (soundness of the eight VCs). *If an MRDT satisfies VCs 1–8, then every configuration reachable from the initial configuration in the ternary transition system is RA-linearizable, per version.*

ra_linearizable_of_core_feasible_cd3

The proof shape: the update layer powers the set-relative convergence theorem, hence canonical states; the transition induction maintains “every store version is canonical for its event set” (fork and query are trivial; apply is the extend lemma); the merge case reduces, via Lemma 3.2, to the ternary Join Lemma, which is proved from VCs 4–8 by strong induction on $|E_1 \cup E_2|$: select a $\text{lo}^{E_1 \cup E_2}$ -maximal e , decompose each side containing e through $\downarrow e$ (LOCAL-REDISTRIBUTE at the honest LCA $\downarrow e \setminus \{e\}$), cancel the shared delta (REDISTRIBUTE), close the principal step with CDVC3, and recurse. The buried-event pathology of Example 3.3 dissolves structurally: no step ever asks how a *side*’s state factors — every factoring happens through the join.

Everything else one might fear owing is proved once, for all data types, as an execution-model invariant: visibility transitivity, per-version backward closure, store well-formedness, and Lemma 3.2 itself. The data type owes exactly the eight equations.

A second route, and an honest asymmetry. One discharged data type does not fit the pattern above. The enable-wins flag’s merge *compares counters* against the LCA, and for such merges the Join Lemma’s hypotheses as stated (backward closure under non-commuting-visibility) are provably too weak: there is a closure-legal tuple — a live LCA enable, a dead post-LCA enable inflating one side’s counter, the killing disable visible only to the other side — on which the production merge computes the wrong flag. Full *causal* closure (which the store invariant actually supplies) excludes it, and the flag is discharged through a full-closure variant of the join. The finding: **required closure strength is a function of the merge’s shape** — commutation-closure suffices for set-shaped merges, counter-comparison merges need full causal closure. Reunifying the two routes under one contract (full closure only strengthens what a data type may assume) is open and, we expect, routine.

MRDT	contract tier	end-to-end theorem	σ -characterization (one line)
Grow-only set	unconditional	<code>goset_ra_linearizable3</code>	union of adds
Grow-only map	unconditional	<code>gomap_ra_linearizable3</code>	union of (k, v) records
Incr.-only counter	unconditional	<code>ioc_ra_linearizable3</code>	<code>#events</code>
PN-counter	unconditional	<code>pn_ra_linearizable3</code>	<code>#inc - #dec</code>
RGA (tombstone)	unconditional	<code>rga_ra_linearizable3</code>	grow-only nodes + tombstones
Peritext	unconditional	<code>peritext_ra_linearizable3</code>	grow-only mark records
OR-set	feasible	<code>ORSet_ra_linearizable3</code>	tag live iff added, no vis-later <code>rem</code>
OR-set (efficient)	feasible	<code>ORSetE_ra_linearizable3</code>	... or same-replica <code>add</code> later (eviction)
Enable-wins flag	full-closure	<code>EWFlag_ra_linearizable3</code>	per key: <code>ctr = #enables</code> ; flag iff unkillable enable
Multi-valued register	feasible	recipe (trichotomy-free: $\text{all-comm} \Rightarrow B = \sigma_0$)	
Add-wins priority queue	feasible	recipe (the OR-set pattern on the add component)	
RGA (tombstone-free)	<i>blocked</i>	feasible <i>update</i> layer needed (below)	

Table 1: The Sal production catalogue against the eight VCs. Nine of twelve carry kernel-checked end-to-end RA-linearizability; all discharges live in a single `MRDT_Instances.lean`.

8 The catalogue, discharged

Table 1 is the empirical heart of the note: the VC set is not merely adequate in principle — the data types people actually ship satisfy it. Three lessons from the sweep:

Tombstones are a metatheoretic trivializer. The catalogue’s celebrated hard cases — the RGA sequence type and the Peritext rich-text type — land in the *easiest* tier. The reason is almost embarrassing: with tombstones, deletion *adds* a record; every state component is grow-only; all operations commute; the merge is a blind join. All of the RGA’s genuine difficulty lives in the *read-side* traversal (turning the node soup into a sequence), which RA-linearizability of the state never touches. By contrast the *tombstone-free* RGA — which physically deletes and repairs positions by climbing ancestor paths — is the one data type the theory cannot yet host, for a reason worth stating precisely: its commutation VC is itself *reachability-conditioned* (two inserts commute only on well-formed states), and our update layer, like the paper’s, quantifies over all states. The missing piece is a feasible *update* layer — update VCs conditioned on an update-and-merge-closed invariant, plus a transport lemma showing every state a canonical enumeration folds through satisfies it. That is the theory’s sharpest open edge, and precisely the question the tombstone-free RGA was built to force.

Commutation class and delta class are independent axes. The multi-valued register is all-commuting, yet *not* in the unconditional tier: its merge is intersection-shaped, $(l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l)$, and fails the delta laws on infeasible tuples exactly as the OR-set does. The boundary between tiers is drawn by *how the merge uses its LCA*, not by whether operations commute. (The compensation: all-commuting makes every punctured causal past empty, $B = \sigma_0$, so the feasible discharge needs no trichotomy at all.)

The eight VCs certify real code on its historical fault line. The enable-wins flag’s repository history contains a known-broken sibling: a global-counter variant whose compositional VC (`inter_right_1op`) fails on DAG-shaped executions, the very defect per-replica counters were introduced to fix. The mechanized σ -facts prove the production merge computes exactly union-liveness — *verified on precisely the corner where the sibling fails*. This is the methodology

working as intended: the metatheory does not merely bless the catalogue, it explains *why* the fixed design is right.

9 Open questions

Open Question 9.1 (CDVC3-derivability). *Is CDVC3 derivable from VCs 1–7 (plus the unconditional delta laws where they hold)? The binary analogue is pinned exactly — there, the corresponding causal-delta bound is equivalent to the Join Lemma and no strictly weaker bridge VC exists — with both attack routes characterized: the positive induction is circular at two identified case shapes, and the countermodel space is narrowed to non-atomic lattice states.*

Open Question 9.2 (structure theorem). *Empirically, the unconditional delta contract holds exactly on the group and lattice classes. Is every unconditional-DeltaVCs3 merge a torsor-like group $\otimes$ lattice combination? (The naïve two-sorted umbrella $\text{mergeL}(l, a, b) = a \boxplus (b \boxminus l)$ does not derive the contract uniformly, so the question is genuinely open.)*

Open Question 9.3 (route reunification). *State the delta laws and CDVC3 over full causal closure so the enable-wins flag’s route becomes an instance of the general one. Full closure only strengthens the discharger’s hypotheses; all current discharges should port unchanged.*

Open Question 9.4 (the feasible update layer). *Condition the update-layer VCs on an update-and-merge-closed invariant (with the permutation-transport lemma along canonical enumerations), and host the tombstone-free RGA. The required invariant machinery ($\text{Rgalnv} \wedge \text{id_mono}$, closed under merge given id-monotone allocation) is already mechanized on the data-type side.*

Open Question 9.5 (the automation layer). *The published catalogue’s seventeen induction-scheme VCs were an SMT-facing Skolemization of feasibility, aimed at the wrong target. Design the analogous per-data-type induction scheme whose induction implies CDVC3 and the feasible delta laws — restoring push-button discharge, now of VCs that actually imply RA-linearizability. The observed uniformity of the discharges (σ -characterization by sandwich invariant + absorber trichotomy) suggests the scheme exists.*

10 Mechanization

Everything above is mechanized in Lean 4 (Mathlib; axioms: `propext`, `Classical.choice`, `Quot.sound`; no `sorryAx` anywhere) in the Sal repository, under `Sal/MRDTs/Metatheory/`:

Artifact	Lean	File
signature, ternary lo	<code>MRDTSig</code> , <code>ConditionedMRDTSig</code>	<code>MRDTSig.lean</code>
store, DAG, transitions	<code>Configuration</code> , <code>Step3</code>	<code>ExecutionModel.lean</code> , <code>LCA_Lemma.lean</code>
Lemma 3.2	<code>lca_events_of_storeInv</code>	<code>LCA_Lemma.lean</code>
σ/lo^E layer (reused binary)	<code>UpdateVCs</code> , <code>IsCanonicalState</code>	<code>Sigma_LoOn3.lean</code>
the eight VCs	<code>CoreVCs3CD</code> , <code>FeasibleDeltaVCs3</code> , <code>CDVC3</code>	<code>VC_Set.lean</code>
Theorem 7.1 + bridges	<code>ra_linearizable_of_core_feasible_cd3</code>	<code>Adequacy.lean</code>
Table 1	all instance theorems	<code>MRDT_Instances.lean</code>
Theorem 3.1, kill-tests	—	<code>Development/Impossibility.lean</code>
findings journal T0–T11	—	<code>Development/MRDT_METATHEORY_DRAFT.md</code>

The binary theory this builds on — including the refutations of the original route, the CD ladder, and the exactness of the causal-delta bound — lives in `Sal/CRDTs/Metatheory/` and

is the subject of the companion note. The production data types of Table 1 are mirrored faithfully from **Sal/MRDTs/*** (encoding deviations documented per instance); their original 24-VC discharges are untouched.